

Data Structures and Algorithms

Assignment 02

Code 1:

```
main() {
```

```
    int sum = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (i > 6)
```

```
            sum = sum + 1;
```

```
        else if (i == 0) {
```

```
            for (int k = 0; k < n; k++) {
```

```
                int y = n;
```

```
                while (y > 0) {
```

```
                    sum = sum - 1;
```

```
                    y--;
```

```
                }
```

```
            }
```

```
        }
```

```
    } else if (i == 2) {
```

this will run int i = 1;

only one while (i < n) {

time, because

due to inner

loop i will be

> n.

```
        int j = 0; 10
```

```
        while (j > 0) {
```

```
            j = j / 2;
```

```
            i = 2 * i;
```

```
        }
```

```
    }
```

```
    }
```

```
    for (int p = 0; p < n; p++) {
```

```
        for (int k = 1; k < n; k *= 2) {
```

```
            cout << "x";
```

```
        }
```

```
    }
```

```
}
```

n = 10

0 → n

} O(1)

0 → n

10 → 0

O(n)

O(n²)

0

1

2

4

8

16

32

64

128

256

512

1024

0

1

2

4

8

16

32

64

128

256

512

1024

n → 1/2⁰

1/2 → 1/2¹

1/4 → 1/2²

1/8 → 1/2³

1/16 → 1/2⁴

1/32 → 1/2⁵

1/64 → 1/2⁶

O(log n)

0 → n

O(log n)

simple find form of code 1

```
for (int i=0; i<n; i++) }  $O \rightarrow n$   
for (int k=0; k<n; k++) }  $O \rightarrow n^2$   
    int y=n;  
    while (y>0) {  
        y--;  
    }  
}
```

```
int i=1;  
while (i<n) { }  $\log n$   
    int j=n;  
    while (j>0) {  
        j/=2;  
        i*=2;  
    }  
}
```

```
for (int p=0; p<n; p++) {  
    for (int k=1; k<n; k*=2) { }  $n \log n$   
}
```

```
{  
    }  $O(n) + O(n^2) + O(\log n) + O(n \log n)$   
    }  $\rightarrow O(n^2)$ 
```



Code 2:

```
void fun(int n) {
    int i, count = 0;
```

```
    if (n > 4) {
```

```
        for (i = 1; i*i <= n; i++)
            count++;
```

```
    } else {
```

```
        point
```

```
        point = 0, i = 1;
        while (i < n) {
```

```
            point++;
            i = 2;
```

```
        } log(n)
```

} log n

```
        for (int y = 1; y < point; y *= 2)
            cout << "We are playing"; } log(log n)
```

$O(\sqrt{n}) + O(\log n) + O(\log(\log n))$

}

Code 3

```
void fun(int n) {
```

```
    int i, j, k, count = 0;
```

```
    for (i = n/2; i <= n; i++) { } n/2 → n
```

```
    for (j = 1; j + n/2 <= n; j++) { } n/2
```

```
    for (k = 1; k <= n; k *= 2) { } log n
```

```
        j + n/2 <= n count++
```

```
        + j ≤ n - n/2 → j ≤ n/2
```

```
    } 10 <= 10
```

}

Code S

```
void fun(int n) {
```

```
    int i, j, k, count = 0;
```

```
    for (i = n/2; i <= n; i++) {  $O(n/2)$ 
```

```
        for (j = 1; j <= n; j = j+1) {  $O(n)$ 
```

```
            for (k = 1; k <= n; k = k*2)  $O(\log n)$ 
```

```
                count++
```

```
        }
```

```
    }
```

```
    int c = 0;
```

```
    for (i = n; i > 0; i /= 2) {  $O(\log n)$ 
```

```
        c++;
```

```
    }
```

```
    for (t = 1; t <= c; t = t*2) {  $\log(\log n)$ 
```

```
        cout << "Hello";
```

```
    }
```

```
}
```

the c we'll get from

upper loop

will be of $\log n$,

beuz loop is

running with the

the complexity

$[O(n) \times O(n) \times O(\log n)] + O(\log n) + O(\log(\log n))$

$O(n^2 \log n)$

~~11~~

Code 6:

```
main() {
```

```
    int s=0;
```

```
    for(int i=0; i<n; i++) {
```

```
        if(i > j) sum+=1;
        else {
```

```
            for(int k=0; k<n; k++) {
```

```
                int y=n;
```

```
                while(y>0) {
```

```
                    sum+=1;
```

```
                    y--;
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
    int i=1;
```

```
    while(i<n) {
```

```
        int j=n;
```

```
        while(j>0) {
```

```
            j/=2;
```

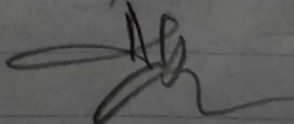
```
            i*=2;
```

```
        }
```

```
    }
```

$\{ O(n) \times O(n) \times O(n) \} + \{ O(\log n) \times O(\log n) \}$

$O(n^3)$



$0 \rightarrow n-1$

$n \rightarrow 1$

So, these all three are nested loops, as else block will execute most of the time

Code 7:

```
void fun1() {
```

```
    cout << "DSA Course";
```

```
    if (n == 1) {
```

```
        for (int i = 1; i <= n; i++) {
```

```
            for (int j = 1; j <= n; j++) {
```

```
                cout << "x";
```

```
                break;
```

```
            }
```

```
        }
```

```
    } else if (i == 10) {
```

```
        for (int p = 0; p < n; p += 10) {
```

```
            for (int y = 0; y < n; y += 2) {
```

```
                count--;
```

```
            }
```

```
        }
```

```
    }
```

but $n = 50$

$p = 0$ $0 < 50$

↓

10 $10 < 50$

↓

20 $20 < 50$

↓

30 $30 < 50$

↓

40 $40 < 50$

↓

50 $50 < 50$

$n/2$

1 → ∞

only one time

0	< 10
↓	
2	< 10
↓	
4	< 10
↓	
6	< 10
↓	
8	< 10
↓	

$n/2$

Time Complexity:

$O(n) + \{ O(\frac{n}{2}) \times O(\frac{n}{10}) \}$

$O(n^2)$

~~11~~

Code 8:

```
void fun(int n) {  
    for (int i = 0; i < n; i++) {  
        for (int j = 1; j < i + i; j++)
```

$0 \rightarrow n-1$

$1 \rightarrow i^2$

```
    }  
}
```

i = 0	, 3	X
i = 1	,	X
2		1, 2, 3
3		1, 2, 3, 4, 5, 6, 7, 8

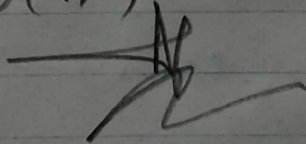
For inner loop:

$i = 0, 1, 2, 3, \dots, n-1$
 $0^2, 1^2, 2^2, \dots, (n-1)^2$

$O(n^2)$

$\hookrightarrow O(n) \times O(n^2)$

$\hookrightarrow O(n^3)$



Question 2: $T(n) = T(n-1) + n$

expanding the recurrence

$$\rightarrow T(n) = T(n-2) + (n-1) + n$$

$$\rightarrow T(n) = T(n-3) + (n-2) + (n-1) + n$$

after k expansions, (we'll hit base case)

$$\rightarrow T(n) = T(n-k) + (n-k+1) + (n-k+2) + \dots + n$$

let's suppose $n-k = 1$ $\therefore 1$ is the base case

$$n-k = 1 \Rightarrow k = n-1$$

Substituting

$$T(n) = T(n-n+1) + (n-n+1+1) + (n-n+1+2) + \dots + n$$

$$\rightarrow T(1) + 2 + 3 + \dots + n$$

$$\rightarrow 2 + 3 + \dots + n$$

$$\rightarrow \frac{n(n+1)}{2} - n$$

So

$$T(n) = T(1) + \frac{n(n+1)}{2} - 1$$

$$\rightarrow O\left(\frac{n^2}{2}\right)$$

$$\rightarrow O(n^2) \quad \underline{\underline{Ans}}$$